Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Kyle Goodwin	kylegood@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea

and a paragraph describing how that paper supports your design concept.
        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider the project below.
        - Create an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the chempy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.

To create a fun and entertaining clicker game

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

The Universal Paperclips Game,

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

This app would be an easy and fun way to procrastinate on work for your classes.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

This program would take in clicks from the user and process that data into money earned and total clicks then return these to the user.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

Clicking on buttons to increase the total clicks and inputting text.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

My program will use a count patterns and if statements to evaluate and manipulate data and outputs.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get back a total count of how many times they have clicked and a total amount of money that they have earned.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

The core things are the clicks that the user does and the money the user has.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Details I need: Lifetime Clicks, Money Earned, Lifetime earnings. Details I will ignore: Clicks per second, active clicking time,

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The automatic clicking part of my idea seems like their could be some possible issues.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

Replace this text with your answer.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

Build something that is a fun little game that I can share with my family and friends.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

Give me the theming for a clicker game, I want a creative idea, like the paperclip game

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

The Receipt That Never Ends: a game where you click to print receipts, as the game goes along you buy upgrades to print more Receipts. *Rubber Duck Takeover: a game where you make rubber ducks, as the game goes along the Rubber Ducks start to take over and make more Rubber Ducks.* *The Missing Sock Machine: Click to create socks, as the game goes along the socks will start forming pairs and make more socks.*

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

The stages seem like they could be hard to make, it seems like it could be a lot of routes and pages to manage. Can you make it down into one or 2 stages?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

Based on this conversation, my project idea has become more focused in adding creative premise to my game vs a very large game.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Office hours,Cs Discord,Bakery LLM.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

Step 1: Read the error and code and see if it is something obvious or if I gave it bad instructions.
Step 2: Feed the error back into the LLM and see if it understands what the issue is.
Step 3 If the LLM doesn't know what is wrong, I will reach out on the Cisc Discord or go to office hours.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

give me the theming for a clicker game
For your clicker game, I'd recommend a neon arcade theme. It would make the game feel satisfying, colorful, and fun while still being realistic to build with Python and Drafter for your CS1 final project.

Theme: Neon Clicker Arcade
Simple Clicker Game — Clicker Game Built with AI | Blink
Neural Stimulator - Clicker Game
67 Clicker - Free Online Idle Clicker Game | Play Now
Color palette
Midnight Background

#100F1F

Electric Purple

#8B5CF6

Neon Cyan

#22D3EE

Hot Pink

#F472B6

Gold Rewards

#FACC15

Card Background

#211D36

Game interface layout
CLICKER GALAXY
TOTAL ENERGY

12,450
+5 energy per click

CLICK!

CLICK TO COLLECT ENERGY

UPGRADES
Power Boost

+1 per click

Cost: 50

Auto Clicker

Passive energy

Cost: 200

LEVEL 3 • KEEP BUILDING YOUR EMPIRE

This is a visual mockup of the style you could recreate in Drafter, not a running game.
Features to include
Main click button: Earn points every time the player clicks.

Upgrade shop: Spend points to increase points per click.

Auto-click upgrade: Gain points automatically over time, if your Drafter implementation supports a suitable timer.

High score: Save the player's best score using browser local storage.

Achievements: Unlock milestones at 100, 1,000, and 10,000 points.

Theme switcher: Let players switch between neon, nature, and candy color palettes.

Make it achievable for CS1
I'd build the game in three stages:

Core gameplay: Score counter, click button, and points-per-click variable.

Progression: Upgrade buttons, costs, and checks to make sure players have enough points.

Polish: Neon colors, score saving, achievement messages, and click animations if supported.

Keep the first version simple: no accounts, multiplayer, server, or database needed.

My recommendation: Use Clicker Galaxy as the name, a dark purple background, a large cyan-outlined click button, gold score text, and pink upgrade headings. That gives you a cohesive design without making the project too complicated.

i want a creative idea, like the paperclip game
Ohhh, you want a clicker game with a weird, oddly specific concept that slowly spirals out of control, like Universal Paperclips — not just a normal game with upgrades.

Here are some more original concepts that could work really well for your Drafter final project.

5 creative clicker game concepts
Cash register with long receipt on blue background vector illustration
1. The Receipt That Never Ends
My favorite
You start by printing one receipt. You earn money for every inch printed. Then you buy faster printers, hire cashiers, open stores, and eventually print receipts for things that don't exist.

The twist: The receipt starts making purchases on its own, and eventually your entire economy exists just to print receipts.

Upgrades: Faster printer, extra paper rolls, cashier army, infinite ink, reality printer.

Surreal Potato Figure in a Dreamlike Landscape | Deep Dream Generator

2. Potato Empire
Click to grow potatoes. Sell them for points, buy farms, genetically modify potatoes, and create potato-powered robots.

The twist: Eventually potatoes become the currency, the workforce, the government, and the entire civilization.

Upgrades: Better soil, potato farms, mutant potatoes, potato AI, planetary potato colonization.

Tarot - Gabriel Kane - Medium
3. Dust Bunny Doomsday
Click to collect dust. Turn dust into bigger dust bunnies, hire tiny vacuum workers, and build a dust empire under your bed.

The twist: Your dust collection operation gets so large that it starts manufacturing the dirt it needs.

Upgrades: Bigger broom, dust farms, lint factories, vacuum army, atmospheric dust generator.

Traumdeutung Badewanne - Traumdeutung.AI
4. Rubber Duck Takeover
Click to create rubber ducks. Sell them, build duck factories, and expand your duck production.

The twist: The ducks start building their own factories, forming duck governments, and colonizing other planets.

Upgrades: Bigger molds, duck assembly line, duck scientists, duck clones, intergalactic duck fleet.

The Existential Crisis of a Lost Sock | by Rita 🌻 | Medium
5. The Missing Sock Machine
Click to generate socks. Collect pairs, research where missing socks go, and build machines that turn single socks into more socks.

The twist: You discover that the missing socks have formed a parallel civilization, and now you must expand into their dimension.

Upgrades: Washing machines, sock cloning, laundry portals, dimension scanners, sock multiverse.

How to make it feel like Paperclips
The secret isn't just the theme. It's the progression:

Stage 1: Simple obsession

Click to make one basic thing and earn points.

Stage 2: Automation

Buy upgrades so the game generates resources by itself.

Stage 3: Unexpected consequences

Unlock strange upgrades that change the story and the purpose of the game.

Stage 4: Absurd escalation

The original task becomes something enormous and ridiculous.

Let's pick your game's identity
Which concept interests you most?

The Receipt That Never Ends

Potato Empire

Dust Bunny Doomsday

Rubber Duck Takeover

The Missing Sock Machine

Give me something even stranger
What kind of atmosphere?

Absurd and funny

Mysterious and unsettling

Cute but increasingly chaotic

Existential and philosophical
How ambitious should it be?

Manageable for a CS1 final project

Ambitious, with multiple unlockable stages
Develop my clicker game concept. Concept: Receipt. Atmosphere: Absurd and funny. Scope: Manageable for CS1. Give me an original premise, progression stages, upgrades, win condition, and a realistic Drafter implementation plan.
The Receipt That Never Ends 🧾
『レシート長すぎ問題』 | ヘザー 落合
Genre: Absurdist incremental clicker / management game

Core idea: You start with a tiny receipt printer at a boring checkout counter. You earn money by printing receipts. Soon, you discover that the longer your receipts get, the more valuable they become. You automate printing, hire employees, and expand your operation until you're printing receipts for purchases that haven't happened, products that don't exist, and eventually the entire universe.

The joke is that your company becomes increasingly successful at solving a problem that nobody has.

1. The premise and story
You are the newly hired employee of Definitely Necessary Receipts, Inc. Your boss gives you one task:

Print receipts. Do not ask why.

At first, it's simple. Click a button to print a receipt and earn a few dollars. But then you unlock upgrades that make the business increasingly ridiculous.

Your first printer prints receipts for actual purchases.

Your second printer prints receipts for the first printer's electricity bill.

Your accounting department starts charging you for the receipts it uses to track receipt expenses.

Your automated system begins printing receipts for future purchases.

Eventually, your company charges the universe a service fee for existing.

The story unfolds through funny messages as the player reaches milestones. You don't need cutscenes or complex animations; changing text and unlocking new upgrades can tell the whole story.

2. Progression: how the game spirals out of control
カスタムPOSレジシステム
Stage 1 — The Humble Cashier
Starting stage
Currency: Dollars

Click PRINT RECEIPT to earn $1. Buy a faster printer to earn more per click.

Story message: "Congratulations! You've printed your first receipt. Nobody asked for it."

Supermarket Price Comparison: An Urgent Review of CHOICE's Methodology and its Consequences for Consumers
Stage 2 — Corporate Nonsense
New mechanic: Automatic printing

Hire an intern and buy extra printers. The business starts earning money without every receipt needing a click.

Story message: "The intern has printed 400 receipts during lunch. Productivity is up 900%. Nobody knows what productivity means."

Byrokratické nesmysly, které zbytečně otravují. Hlasujte pro absurditu roku - Médium.cz
Stage 3 — The Receipt Economy
New mechanic: Absurd business upgrades

You start printing receipts for the cost of printing receipts. Your company has departments, fees, and increasingly meaningless products.

Story message: "The accounting department has invoiced itself. The company is now both its best customer and its worst customer."

¿Qué se celebra el 7 de mayo?, efemérides, sucesos históricos, nacimientos y muertes - Infobae
Stage 4 — The Infinite Receipt
Final mechanic: Universal printing

Your printers issue receipts for future events, imaginary purchases, and the existence of the universe.

Story message: "The universe has received its receipt. Please retain it for warranty purposes."

For a CS1 project, these stages can simply unlock at score milestones. You don't need to build four separate game systems.

3. Upgrades the player can buy
Here is a balanced starter shop. The costs are suggested starting values, so you can adjust them during testing.

Upgrade

Cost	Effect
Slightly Better Printer	$10	+1 dollar per click
Suspiciously Long Paper	$50	+5 dollars per click
Unpaid Intern	$100	+1 dollar per automatic tick
Printer Farm	$250	+5 dollars per automatic tick
Accounting Department	$750	+20 dollars per automatic tick
Time-Travel Receipt	$2,000	+100 dollars per automatic tick
Universal Paper Roll	$10,000	Unlock the final stage

The humor comes from the upgrade names and descriptions, not just the numbers.

For example, the Time-Travel Receipt description could say: "Collect payment for purchases that haven't happened yet. Legal says this is probably fine."

You can make each upgrade purchasable multiple times, with its price increasing after every purchase. That gives players a reason to keep earning money without requiring dozens of unique items.

4. What makes the game fun beyond clicking?
I'd add three small features that create personality without making the code too complicated.

Random corporate messages

After a purchase or milestone, display a funny message such as "Your printer has requested paid vacation."

Company statistics

Track total receipts printed, total money earned, and upgrades purchased. These make progress feel meaningful.

Milestone announcements

Unlock titles such as "Paper Pusher," "CEO of Nothing," and "Supreme Receipt Officer" at specific totals.

5. Win condition
The game should have a real ending, even if it feels ridiculous.

Final goal: Print the Receipt for Everything.

To unlock the ending, the player must:

Earn $10,000,000 in lifetime revenue.

Purchase the Universal Paper Roll.

Reach the final story milestone.

The ending message:

TRANSACTION COMPLETE
Item: The Entire Universe

Subtotal: $9,999,999.99

Administrative Fee: $0.01

TOTAL: $10,000,000.00

THANK YOU FOR YOUR EXISTENCE.
Please keep this receipt. It is required to prove that reality occurred.

No refunds. The universe is final sale.

After winning, let the player view their final statistics and optionally keep playing in an endless mode.

6. A realistic Drafter implementation plan
Because Drafter runs Python in the browser through Pyodide, you can build this as a self-contained game. You don't need a server, database, user accounts, or an online leaderboard.

Part 1 — Core clicker

Essential
A button that prints a receipt.

Variables for money, receipts printed, and money per click.

A text display showing the current balance.

A simple upgrade that costs money.

Part 2 — Shop and progression

Essential
A list of upgrades with prices and effects.

A check that prevents purchases when the player lacks money.

Increasing upgrade costs.

Story messages and stage unlocks based on milestones.

Part 3 — Persistence and polish

Stretch goal
Save progress in browser local storage.

Add a reset button with confirmation.

Style the interface like a ridiculous corporate receipt.

Add a final winning screen and statistics.

The main game variables
You can represent most of the game with a handful of basic Python variables:

Variable

Purpose

money

Current spendable money

total_earned

Lifetime money earned

receipts_printed

Total receipts created

money_per_click

Money earned per manual click

auto_income

Money earned per automatic tick

upgrades

Counts of purchased upgrades

game_stage

Current stage of the story

The main processing logic is straightforward:

The player clicks the print button.

Add money_per_click to money and total_earned.

Increase receipts_printed.

Check whether the player has unlocked a new stage.

Update the page to show the new values.

For purchases, check that the player has enough money before subtracting the cost and applying the upgrade.

One important technical choice: automatic income needs a timer that works with your Drafter setup. To keep the project manageable, you can start with manual clicking and add passive income only after the basic game works. If continuous timers prove difficult, an alternative is to let the player click a button to process a batch of automatic receipts.

For saving, use browser local storage if you want progress to persist after refreshing the page. Keep the game's state in Python while the game is running, and test how it is restored when the page reloads.

7. The visual identity
『レシート長すぎ問題』 | ヘザー 落合
IT-Lösungen & Sicherheit, zuverlässig und verständlich | Koschier IT Outsourcing GmbH
The Final Boss (itch) (Cakestorm) - release date, videos, screenshots, reviews on RAWG
I'd avoid the typical neon clicker aesthetic for this concept. Make it look like an ugly, soulless corporate checkout system that gradually becomes absurd.

Background: warm receipt-paper beige (#F5F0E6)

Text: printer-black (#242424)

Primary button: dark forest green (#285943)

Warning accents: faded red (#C65A46)

Font: a monospace font to imitate printed receipts

Layout: a receipt-style header, a huge money counter, a prominent print button, and a shop panel

As the game progresses, the receipt shown on screen can become longer, displaying the player's latest achievements, purchases, and increasingly ridiculous fees. You can implement this as ordinary text rather than generating actual paper graphics.

My final recommendation
Build the game around one memorable joke: you are creating an enormous business whose only product is proof that the business exists.

That gives you a creative premise, a reason for upgrades, a natural progression system, plenty of opportunities for funny writing, and a clear win condition—all achievable with basic Python variables, conditionals, lists, functions, and Drafter page updates.

the stages seem like they could be hard to make, it seems like it could be a lot of routes and pages to manage
You're right — and honestly, I would simplify it. For a CS1 project in Drafter, you don't need separate pages for each stage. You can make the game feel like it's evolving while keeping everything on one page.

My recommendation: One page, one increasingly ridiculous receipt
TOTALLY NORMAL RECEIPTS INC.
OFFICIAL BUSINESS TRANSACTION

YOUR COMPANY BALANCE

$1,250
Lifetime receipts printed: 342

PRINT RECEIPT
UPGRADES
Better Printer

Earn more per click

$50

Unpaid Intern

Prints receipts automatically

$100

LATEST COMPANY MEMO

"Management has approved printing receipts for the receipts used to print receipts."

Concept mockup — the entire game can use this single-page layout.
How to make stages work without creating more pages
Instead of making each stage a new page, make the stages milestones that change the content on the same page.

When the player reaches…

What changes

$100

A new funny company memo appears

$1,000

The next upgrade becomes available

$10,000

The company name changes to "Receipt Conglomerate"

$100,000

The game unlocks absurd new upgrade descriptions

$1,000,000

The final objective appears

These are just if statements that check the player's money or lifetime earnings. The same buttons, shop, and layout remain in place.

Keep the Python structure simple
You could organize the game around just four main functions:

print_receipt(state) — adds money and increments the receipt counter.

buy_upgrade(state, upgrade_name) — checks affordability and applies an upgrade.

get_company_memo(state) — returns a funny message based on progress.

render(state) — displays the current balance, upgrades, memo, and receipt history.

You'd keep the game data in a single state object, rather than building a separate route for every feature.

One important detail: in Drafter, you'll want button handlers to update the state and return the same game page. Follow the state and page patterns supported by your course's Drafter version.

What I'd actually build for your final project
Start with just:

One page.

One print button.

Three upgrades.

A money counter and lifetime receipt counter.

Five or six funny milestone messages.

A win condition based on lifetime earnings.

Add local storage or automatic income only after that works.

The key design principle: the game should become more absurd, not more complicated to navigate. The player experiences the growing madness through the numbers, upgrade names, and company memos—not by moving between pages.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.